Group 2: CourseLens

Manager: Rayan Chahid

Members: Le, Brendan, Isaac, Zion, Sreshta

Frameworks Used

Backend:

- Next.js
- PostGres with Supabase
- Data management with Python/Flask to and from SupaBase
  - Data scraping and checking
  - Calculating metrics

Frontend:

- React
- Next.js
- Plotly for graphs
- Github hosted site

End Point Routes

User info

GET / → home page	
GET /signup → signup page (user, pass)	POST /signup → try to register user
GET /login → login page (user, pass)	POST /login → try login
GET /eval → class evaluation page (class, score, difficulty...)	POST /eval → Upload evaluation. Call is routed to internal addCourse/addProfessor calls if needed.

Class information internals GET

GET /course/[courseID] ➔ calls getCourseData(courseID)	Show all metrics for a course
GET /api/course/[courseID]/professors ➔ calls getCourseData(courseID, professor)	Retrieve professor-specific data for a course
GET/api/course/[courseID]/metrics ➔ calls computeGradeDistribution() and computeAvgGPA()	Compute/Retrieve data for a course

Class information internals POST

POST /api/course/add ➔ calls addCourse(courseID, name, grades)	User's evaluation is on a new course, so add the course
POST /api/course/[courseID]/professors ➔ calls addProfessor(courseID,professorName)	User's evaluation includes a new professor, add this professor
POST /api/course/[courseID]/professors ➔ calls addProfessor(courseId, professorName, grades)	Evaluation includes professor, links eval to prof

Tech Stack

System Architecture Overview

1- User Interaction with React.js

2- next.js Application

3- Supabase (Postgres Database + API)

4- JSON Data

5- Plotly Visual

6- Data processing in python (pull from supabase, compute, push processed data back to supabase) + Data scraping and verification

System Architecture - User and Data Interaction

1- User Interaction

- Use selects a course , Ex: CS 220
- Optional Filter by Professor Davila

2- Data Fetching -

- Supabase filters data in PostgreSQL
- returns structured JSON

```
let query = supabase.from("courses").select("*").eq("course_id", 220)

if (professor) {
  query = query.eq("professor", "Davila")
}

const { data, error } = await query
```

Data Processing

- Python pulls from supabase
- Aggregates grades
- Computes average GPA
- Structures data for visualization
- Pushes processed data back into SupaBase
- Checks data to make sure nothing is strange (not adding a class ‘CS6767’)
- Web scraping to get lists of classes and teachers


```
{
  "grades": { "A": 45, "B": 30, "C": 12, "D": 5, "F": 3 },
  "average_gpa": 3.21
}
```

System Architecture - Visualization

Visualization (Plotly)

- Data is passed into plotly
- Bar chart for grade distribution
- Line chart for enrollment
- Percentage “Fit” score for a person
- Average GPA

Function Signatures

Data Access

- getCourseData(courseID: number, professor?: string): Record<string, number>

Analytics

- computeGradeDistribution(courseID: number): Record<string, number>
- computeAvgGPA(data: Record<string,number>): number

Visualization

- GradeDistributionPlot(data: Record<string,number>): JSX.Element

Write Functions

addCourse(courseID: number, name: string): void

addProfessor(courseID: number, professorName: string): void

How we will Test-

Framework: Jest — run via GitHub CI on each commit (npm test)

- To test the frontend and data retrieval. Examples:

getCourseID() filters by professor: Mock Supabase to return 3 rows for CS 220. Call getSections(220, "Davila"). expect(result).toHaveLength(1) and expect(result[0].professor).toBe("Davila").

GET /api/course/[courseID] returns 200: Use jest-fetch-mock to mock Supabase. Call handler with courseID=220. expect(res.status).toBe(200) and expect(body.grades).toBeDefined()

- To test backend and data processing (unittest, python, mock requests, and test push to SupaBase. Examples:

computeAvgGPA() returns correct value: Pass mock sections with grades {A:45, B:30, C:12, D:5, F:3}. expect(computeAvgGPA(sections)).toBeCloseTo(3.21).

computeGradeDistribution() sums correctly: Pass 2 mock sections each with {A:10, B:5}. expect(result).toEqual({A:20, B:10}).

POST /api/eval stores evaluation: Mock Supabase insert. POST payload {courseId:220, rating:5}. expect(supabase.from("evals").insert).toHaveBeenCalledWith({courseId:220, rating:5}).

How we will Test-

Framework: Playwright for UI testing and visualization (https://playwright.dev/)

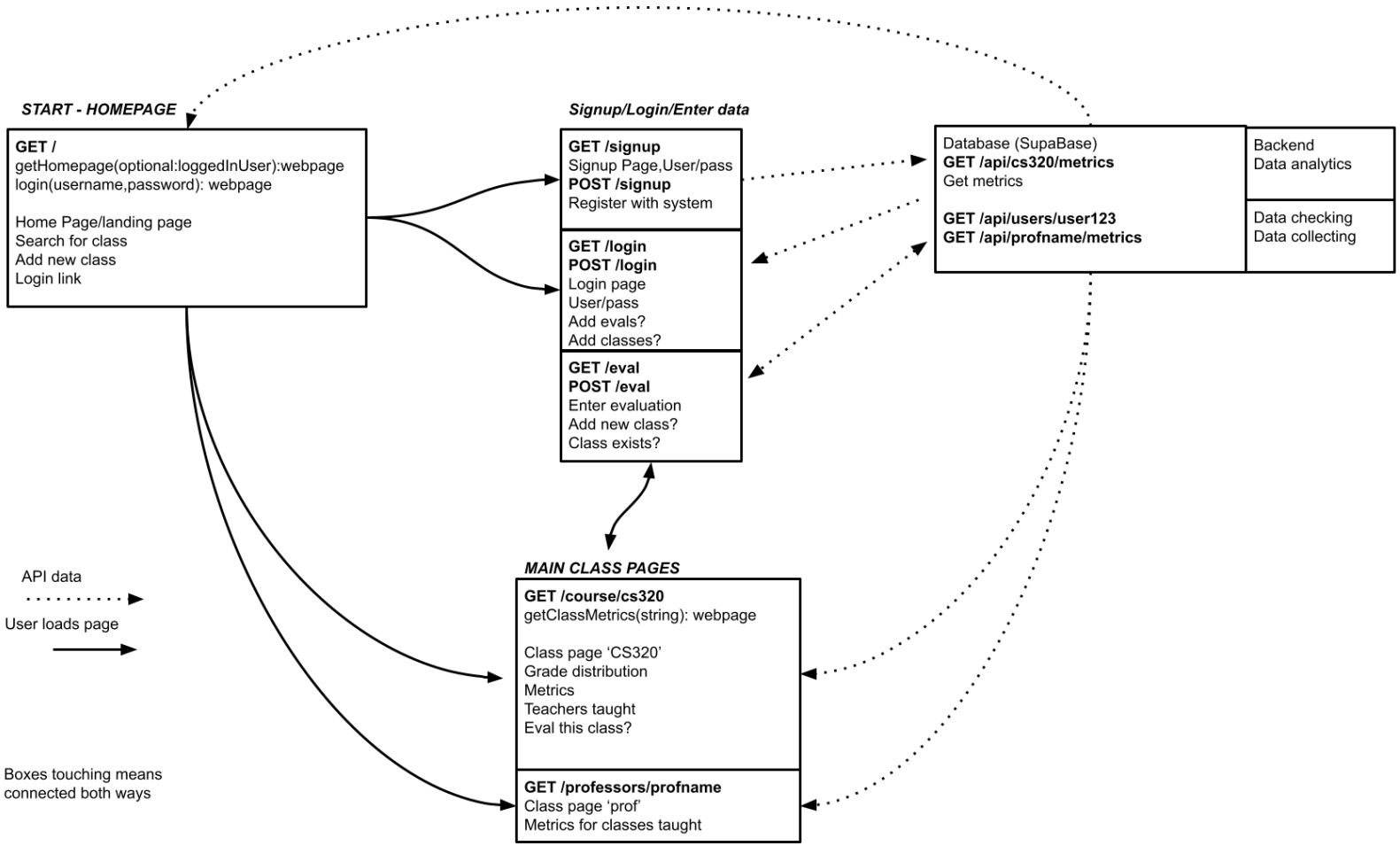
- To test Plotly visuals

GradeDistributionPlot() works for an example class

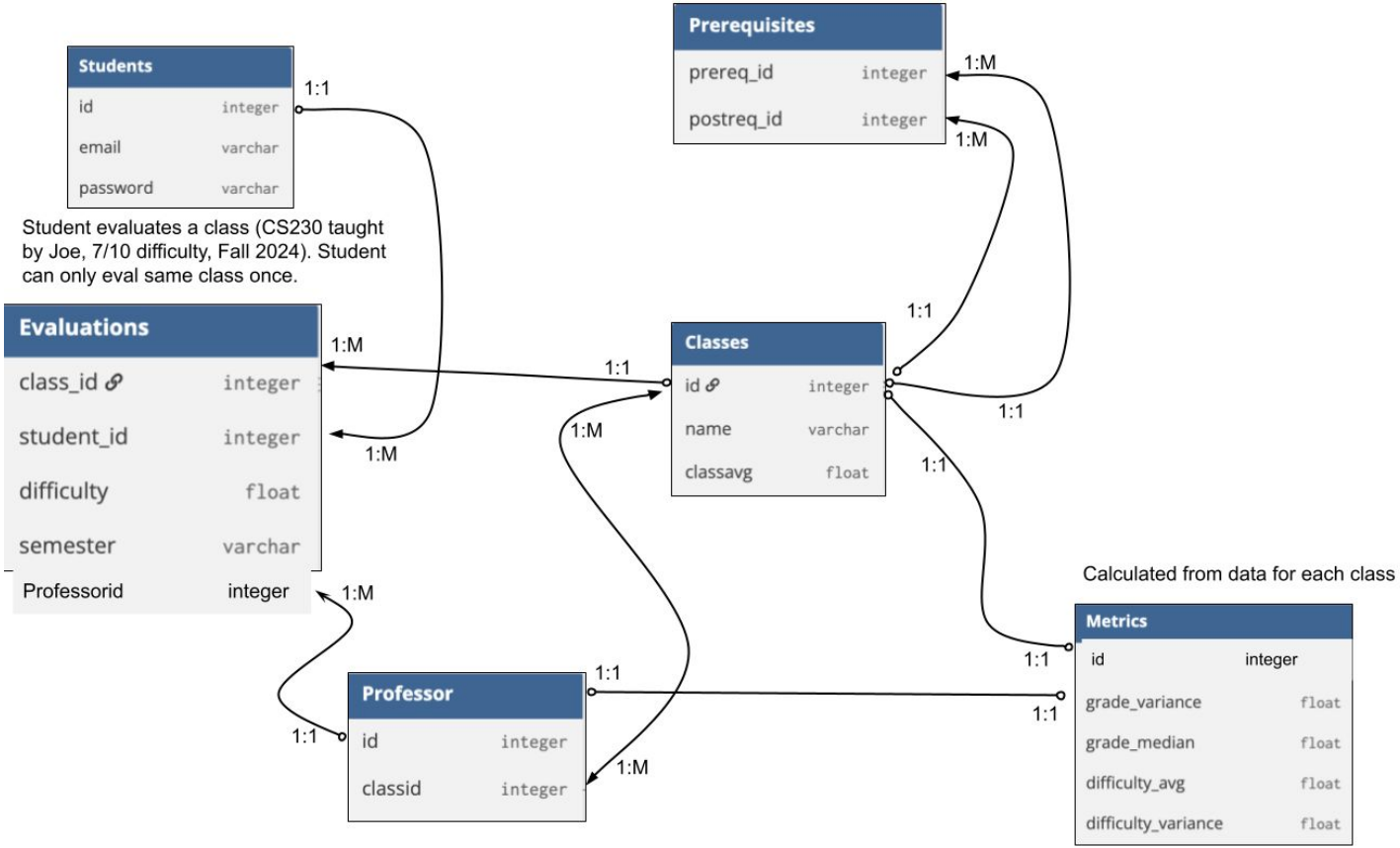
- Test UI redirection, login, buttons all work

Sign up -> login -> add eval -> open class -> open prof. page

THANKS!


START - HOMEPAGE
GET /
getHomepage(optional:loggedInUser):webpage
login(username,password): webpage
Home Page/landing page
Search for class
Add new class
Login link
Signup/Login/Enter data
GET /signup
Signup Page,User/pass
POST /signup
Register with system
GET /login
POST /login
Login page
User/pass
Add evals?
Add classes?
GET /eval
POST /eval
Enter evaluation
Add new class?
Class exists?
Database (SupaBase)
GET /api/cs320/metrics
Get metrics
GET /api/users/user123
GET /api/profname/metrics
Backend
Data analytics
Data checking
Data collecting
MAIN CLASS PAGES
GET /course/cs320
getClassMetrics(string): webpage
Class page 'CS320'
Grade distribution
Metrics
Teachers taught
Eval this class?
GET /professors/profname
Class page 'prof'
Metrics for classes taught
API data
User loads page
Boxes touching means
connected both ways

Student Signs up for courselens



Student evaluates a class (CS230 taught by Joe, 7/10 difficulty, Fall 2024). Student can only eval same class once.

Calculated from data for each class

We will need a 'professor' page -> which shows classes taught, the metrics for those classes. Note there is NO aggregated professor metrics, only class specific ones.

Class inheritance for Metrics ?
Proff. Metrics and Class. Metrics can inherit from metrics if possible to allow for easier Id'ing